

Online Bin Covering Algorithm — Usage Guide

Mia Miletić, Anja Pijak

Requirements

The solution is written in **Python 3** and requires no external libraries. Any Python version ≥ 3.6 is sufficient.

Archive Contents

File	Description
<code>solve.py</code>	Main solver
<code>checker.py</code>	Validates a solution against the input
<code>gen_inputs.py</code>	Generates random sample input files
<code>input10.txt</code>	Sample input: 10 items
<code>input100.txt</code>	Sample input: 100 items
<code>input500.txt</code>	Sample input: 500 items
<code>input600.txt</code>	Sample input: 600 items
<code>input700.txt</code>	Sample input: 700 items
<code>output10.txt</code>	Corresponding output for <code>input10.txt</code>
<code>output100.txt</code>	Corresponding output for <code>input100.txt</code>
<code>output500.txt</code>	Corresponding output for <code>input500.txt</code>
<code>output600.txt</code>	Corresponding output for <code>input600.txt</code>
<code>output700.txt</code>	Corresponding output for <code>input700.txt</code>

Input Format

The input file must follow this format:

- Line 1: a single integer n — the number of items.
- Lines 2 to $n + 1$: one item weight per line, a real number in $(0, 1]$.

Example (`input10.txt` excerpt):

```
10
0.413271
0.783158
0.278345
...
```

Output Format

The output file produced by the solver follows this format:

- Line 1: a single integer k — the number of covered bins.
- Lines 2 to $k + 1$: space-separated 1-indexed item indices assigned to each bin.

Example:

```
3
2 5 7
1 4
3 6 8
```

Running the Solver

```
python solve.py <input_file> <output_file>
```

If no arguments are given, defaults are `input.txt` and `output.txt`.

Examples:

```
python solve.py input100.txt output100.txt
python solve.py                        # uses input.txt -> output.txt
```

Checking a Solution

The checker verifies that a solution file is valid for a given input:

```
python checker.py <input_file> <output_file>
```

It checks:

- Every item index is in the valid range $[1, n]$.
- No item is used more than once.
- The sum of weights in each reported bin is ≥ 1 .

On success it prints OK along with the number of covered bins and unused items. On failure it prints ERROR with a description and exits with code 1.

Example:

```
python checker.py input100.txt output100.txt
# OK: 41 valid covered bins, 0 items unused
```

Generating Sample Inputs

To regenerate the input files, add to the `gen_inputs.py` line of code:

```
gen(<n>, "input<n>.txt", seed=42)
```

Then run:

```
python gen_inputs.py
```